TECHNICAL STANDARD OPERATING PROCEDURE

Azure Database for PostgreSQL

Flexible Server — Database Administration

Comprehensive administration notes and operational reference spanning provisioning, security, resilience, performance, and governance

Document type	Technical SOP / Reference Notes
Platform	Azure Database for PostgreSQL — Flexible Server
Audience	Cloud Database Administrators
Author	PMSQLDBA
Version	1.0
Date	2026-07-09

Table of Contents

Part I — Foundations 4
1. Service Overview and Architecture 4
1.1 Architecture model 4
1.2 Resource hierarchy 4
1.3 Supported versions 4
2. Provisioning and Deployment 4
2.1 Create a server (Azure CLI) 5
2.2 Compute tiers 5
2.3 Storage rules to memorise 5
Part II — Connectivity and Security 6
3. Networking and Connectivity 6
3.1 Access models 6
3.2 Connecting with psql 6
4. Security and Encryption 6
4.1 Encryption at rest 6
4.2 Encryption in transit 6
4.3 Advanced isolation 6
5. Authentication and Access Control 7
6. Roles, Users and Permissions 7
6.1 Creating roles and granting privileges 7
6.2 Least-privilege design 8
7. Database and Object Administration 8
8. Server Parameters and Configuration 8
8.1 Key tuning parameters 9
9. Extensions 9
Part III — Resilience and Recovery 9
10. Backup, Restore and Recovery 9
11. High Availability 10
12. Replication and Read Scaling 10
13. Disaster Recovery 11
Part IV — Operations 11
14. Scaling 11
15. Connection Management 11

16. Performance Monitoring and Tuning....12
16.1 Tools....12
16.2 Common tuning workflow....12
17. Monitoring, Logging and Alerting....12
18. Maintenance and Upgrades....13
Part V — Lifecycle and Governance....13
19. Migration....13
20. Cost Management and Optimization....13
21. Automation and Infrastructure as Code....14
22. Compliance and Governance....14
23. Advanced and Emerging Capabilities....14

Part I — Foundations

1. Service Overview and Architecture

Azure Database for PostgreSQL flexible server is the current managed PostgreSQL Platform-as-a-Service (PaaS) offering on Azure. It runs the community PostgreSQL engine (no fork), so applications migrate with minimal refactoring. Microsoft owns the hardware, operating system, and engine binaries; the DBA owns configuration, security, data, and workload design.

> **KEY POINT** Flexible server is the only model to learn. The legacy Single Server model is retired — do not build new workloads on it.

1.1 Architecture model

- Compute and storage are decoupled. The engine runs in a container on a Linux VM; data files live on Azure storage.
- Storage keeps three locally-redundant synchronous copies of every data file, giving baseline durability before any HA is configured.
- Because compute is separate, vCores scale up or down in seconds without moving data.

1.2 Resource hierarchy

Level	Meaning
Subscription	Billing and top-level ownership boundary.
Resource group	Logical container for the server and related resources.
Server (instance)	The administrative unit; parent of all databases. Deleting it deletes every contained database.
Database	Created inside a server; one server can host one or many databases.

Two databases exist at creation: **postgres** (default connection target) and **azure_maintenance** (managed-service internal — no user access).

1.3 Supported versions

The service tracks community PostgreSQL major versions across the 13–18 range, with in-place major version upgrades and automatic minor-version patching. Always confirm the current supported list in the Azure docs before planning an upgrade.

2. Provisioning and Deployment

A server can be deployed through the Azure portal, Azure CLI, PowerShell, ARM/Bicep, Terraform, or the REST API. CLI is the fastest repeatable path for a DBA.

2.1 Create a server (Azure CLI)

```
# Login and set subscription
az login
az account set --subscription "<SUBSCRIPTION_ID>"

# Create a flexible server
az postgres flexible-server create \
  --resource-group rg-pgsql-prod \
  --name pg-prod-eastus2 \
  --location eastus2 \
  --tier GeneralPurpose \
  --sku-name Standard_D4ds_v5 \
  --storage-size 128 \
  --version 16 \
  --admin-user pgadmin \
  --admin-password "<STRONG_PASSWORD>" \
  --high-availability ZoneRedundant \
  --tags env=prod owner=dba
```

2.2 Compute tiers

Tier	Use case
Burstable	Dev, test, lab, low-concurrency workloads. Accrues CPU credits while idle. Lowest cost.
General Purpose	Balanced production OLTP requiring predictable performance and concurrency.
Memory Optimized	Buffer-heavy / high-concurrency production workloads needing more memory per vCore.

NOTE A newer V6 SKU family (General Purpose / Memory Optimized) is in preview for high-scale OLTP and analytics. Scaling between V6 and Burstable is not supported in either direction.

2.3 Storage rules to memorise

- Storage scales UP only — it can never be decreased in place.
- Storage grows in 2x increments and scaling up requires a server restart.
- To shrink storage, dump and restore into a new, smaller server.

WARNING Size storage deliberately at creation. The only path to a smaller footprint is a full dump-and-restore migration.

Part II — Connectivity and Security

3. Networking and Connectivity

3.1 Access models

Model	Description	When to use
Private access (VNet)	Server injected into a delegated subnet with a private DNS zone. No public endpoint.	Production, compliance-sensitive workloads.
Public access	Public endpoint gated by firewall / allowed-IP rules.	Quick dev/test or where private networking is not required.
Private Link	Private endpoint exposing the server inside your VNet address space.	Hub-spoke topologies, cross-VNet private access.

WARNING The private-access networking choice is fixed at creation and cannot be switched later without recreating the server. Decide VNet vs public before deployment.

3.2 Connecting with psql

```
psql "host=pg-prod-eastus2.postgres.database.azure.com \
      port=5432 dbname=postgres user=pgadmin \
      sslmode=require"
```

Clients: psql, pgAdmin, Azure Data Studio, and any standard libpq / driver. Port 5432 for direct connections; port 6432 for the built-in PgBouncer pooler.

4. Security and Encryption

4.1 Encryption at rest

- AES-256 storage encryption using a FIPS 140-2 validated cryptographic module.
- Covers data, backups, and temporary files created while running queries.
- Keys are system-managed by default; customer-managed keys (CMK / BYOK) via Azure Key Vault are supported for regulatory control.

4.2 Encryption in transit

- SSL/TLS is enforced by default; the service supports TLS 1.2 and later.
- Use sslmode=require (or stricter verify-full with the CA cert) on all clients.

4.3 Advanced isolation

Azure Confidential Computing allows flexible server instances to run on Confidential VMs inside a Trusted Execution Environment (TEE) for workloads requiring hardware-level memory isolation.

> **KEY POINT** Defense in depth: private networking + CMK at rest + TLS in transit + least-privilege roles. Layer them; do not rely on any single control.

5. Authentication and Access Control

Method	Notes
PostgreSQL native (password)	Traditional role + password authentication.
Microsoft Entra ID	Azure AD identities and groups authenticate to the database; centralised, supports MFA and group-based access.
Managed identity	System- or user-assigned identity for passwordless app-to-database connections.

> **NOTE** Both native and Entra ID authentication can be enabled together. Prefer Entra ID for human and app access to centralise identity and eliminate stored passwords where possible.

6. Roles, Users and Permissions

The admin account created at provisioning is the highest-privilege user but is **NOT a full PostgreSQL superuser**. It belongs to the **azure_pg_admin** role. The true superuser attribute is held by **azure_superuser**, owned by the managed service.

> **WARNING** Commands that assume SUPERUSER on a self-hosted install may fail. Manage extensions, roles, and parameters through the Azure-exposed interfaces rather than expecting god-mode.

6.1 Creating roles and granting privileges

```
-- Create a login role
CREATE ROLE app_rw LOGIN PASSWORD '<pwd>';

-- Create a group (no login) and add members
CREATE ROLE readers;
GRANT readers TO app_rw;

-- Grant object privileges
GRANT CONNECT ON DATABASE salesdb TO app_rw;
GRANT USAGE ON SCHEMA public TO app_rw;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO app_rw;

-- Ensure future tables inherit the grant
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT ON TABLES TO readers;
```

6.2 Least-privilege design

- Grant to group roles, then add users to groups — never grant directly to individuals.
- Separate read-only, read-write, and DDL roles per application.
- Use ALTER DEFAULT PRIVILEGES so new objects inherit correct grants automatically.
- Apply Row-Level Security (RLS) for multi-tenant data isolation where required.

7. Database and Object Administration

```
CREATE DATABASE salesdb;
CREATE SCHEMA reporting AUTHORIZATION app_rw;

-- Range partitioning example
CREATE TABLE orders (
  id bigint, order_date date, amount numeric
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2026_q1 PARTITION OF orders
  FOR VALUES FROM ('2026-01-01') TO ('2026-04-01');
```

- Objects: tables, indexes, views, materialized views, sequences, constraints.
- Partitioning: range, list, and hash — automate creation/pruning of partitions with pg_cron.
- Large values are handled transparently via TOAST; be aware of it when tuning wide rows.

> **TIP** Schedule partition maintenance (create next period, detach/drop old) with pg_cron so it runs inside the database on a fixed cadence.

8. Server Parameters and Configuration

Flexible server exposes a managed subset of postgresql.conf. View and edit parameters via portal or CLI. Some are dynamic (apply immediately); others are static and require a restart.

```
# List and set a parameter
az postgres flexible-server parameter set \
  --resource-group rg-pgsql-prod --server-name pg-prod-eastus2 \
  --name work_mem --value 16384

# Show a parameter
az postgres flexible-server parameter show \
  --resource-group rg-pgsql-prod --server-name pg-prod-eastus2 \
  --name shared_buffers
```

8.1 Key tuning parameters

Parameter	Purpose	Restart?
shared_buffers	Engine cache; typically ~25% of RAM.	Yes
work_mem	Per-operation sort/hash memory.	No
effective_cache_size	Planner hint for OS+DB cache size.	No
max_connections	Concurrent connection ceiling (prefer PgBouncer over raising).	Yes
autovacuum_*	Controls vacuum aggressiveness and bloat control.	No
wal_* / checkpoint_*	Write-ahead log and checkpoint behaviour.	Mixed

9. Extensions

Extensions must be allow-listed before use. Add them to the allow-list, then CREATE EXTENSION.

```
# Allow-list via azure.extensions parameter, then in SQL:
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
CREATE EXTENSION IF NOT EXISTS pgvector;
CREATE EXTENSION IF NOT EXISTS pg_cron;
```

Extension	Use
pg_stat_statements	Query performance statistics (slow-query analysis).
PostGIS	Geospatial data types and queries.
pgvector	Vector similarity search for AI / embeddings.
pg_cron	In-database job scheduler.
pgAudit	Fine-grained audit logging for compliance.
TimescaleDB	Time-series workloads.
pg_duckdb	Analytical (OLAP) acceleration inside Postgres (preview).

Part III — Resilience and Recovery

10. Backup, Restore and Recovery

- Backups are automatic and stored on zone-redundant storage (ZRS).
- Configurable retention window; on-demand backups supported.
- Point-in-time restore (PITR) creates a NEW server at a chosen timestamp within the retention window.
- Long-Term Retention (LTR) keeps backups up to 10 years for audit/regulatory needs.
- Geo-redundant backup enables geo-restore into a paired region.

```
# Point-in-time restore to a new server
az postgres flexible-server restore \
  --resource-group rg-pgsql-prod \
  --name pg-prod-restored \
  --source-server pg-prod-eastus2 \
  --restore-time "2026-07-08T09:30:00Z"
```

KEY POINT PITR and geo-restore always create a new server. Plan connection-string cutover and firewall/VNet re-configuration as part of any restore runbook.

11. High Availability

Mode	Behaviour
Zone-redundant HA	Warm standby in a different AZ, synchronous replication, zero data loss. Highest resilience.
Same-zone HA	Standby in the same AZ — protects against node failure with lower latency, less regional protection.
No HA	Single node; relies on three-copy storage durability and backups only.

- On failover the standby comes online immediately to process transactions.
- Near-zero-downtime scaling and failover keep the connection string stable (typically under ~30s).
- Billing: zone-redundant HA bills compute + storage for BOTH primary and standby.

NOTE HA protects availability, not human error. It replicates a bad DELETE instantly — backups/PITR remain your data-loss safety net.

12. Replication and Read Scaling

- Read replicas (in-region and cross-region) offload read traffic; cascading replicas chain from a replica.
- Logical replication (publications/subscriptions) enables selective, row-level data movement and near-zero-downtime migrations.
- Monitor replication slots and lag; watch the logical_replication_slot_sync_status metric for failover readiness.

```
# Create an in-region read replica
az postgres flexible-server replica create \
  --replica-name pg-prod-replica1 \
  --source-server pg-prod-eastus2 \
  --resource-group rg-pgsql-prod
```

13. Disaster Recovery

- Design RPO/RTO targets first, then choose mechanisms: geo-redundant backup (geo-restore), cross-region read replica (promote on failover).
- Document a regional-failover playbook: promote replica, repoint application, validate, and re-establish DR in the new region.
- Test DR on a schedule — an untested DR plan is an assumption, not a plan.

> **TIP** Cross-region read replica gives the lowest RTO for regional outage: promote the replica and cut over, rather than waiting on a geo-restore.

Part IV — Operations

14. Scaling

- Vertical: change vCores up/down or switch tier. On HA-enabled servers this is near-zero-downtime.
- Storage: scales up only, in 2x increments; enable storage autogrow to avoid running out of space.
- Connections: scale via PgBouncer rather than raising max_connections.
- Horizontal: Elastic clusters shard across a cluster of nodes for very large / high-throughput workloads.

```
# Vertical scale (change SKU)
az postgres flexible-server update \
  --resource-group rg-pgsql-prod --name pg-prod-eastus2 \
  --sku-name Standard_D8ds_v5
```

15. Connection Management

Every connection — idle or active — consumes real resources and can cause CPU, disk, and lock contention. The server sets a fixed max_connections from the chosen compute and reserves ~15 connections for replication and monitoring.

- Use the built-in PgBouncer pooler (same hostname, port 6432) in transaction mode.
- Start pool sizing at vCores × 2–5, then tune against observed utilisation.
- "FATAL: sorry, too many clients already" means the connection ceiling is hit — pool, do not just raise the limit.

> **KEY POINT** Default reflex for connection pressure is PgBouncer in transaction mode — not increasing max_connections, which consumes memory per slot and worsens contention.

16. Performance Monitoring and Tuning

16.1 Tools

- Query Store / Query Performance Insight — captures top queries and their resource cost.
- pg_stat_statements — aggregate query statistics for slow-query hunting.
- Autonomous tuning — ML-driven recommendations for a subset of server parameters.
- Azure Advisor — proactive index, logging, statistics, storage, and connection recommendations.

16.2 Common tuning workflow

```
-- Top time-consuming queries
SELECT query, calls, total_exec_time, mean_exec_time
FROM pg_stat_statements
ORDER BY total_exec_time DESC LIMIT 20;

-- Check table bloat / last autovacuum
SELECT relname, n_dead_tup, last_autovacuum
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC;
```

- Index tuning: add missing indexes flagged by index-scan insights; drop unused ones.
- Autovacuum: tune per-table thresholds on high-churn tables to control bloat.
- Run ANALYZE after bulk loads so the planner has fresh statistics.

17. Monitoring, Logging and Alerting

- Azure Monitor metrics (CPU, memory, IOPS, storage, connections, replication lag) with metric alerts.
- Diagnostic settings route server and audit logs to Log Analytics, Storage, or Event Hubs.
- Built-in Grafana dashboards for at-a-glance server health.
- pgAudit for statement-level audit trails feeding compliance evidence.

```
# Route logs to a Log Analytics workspace
az monitor diagnostic-settings create \
  --name pg-diag \
  --resource <SERVER_RESOURCE_ID> \
  --workspace <LOG_ANALYTICS_ID> \
  --logs '[{"category":"PostgreSQLLogs","enabled":true}]' \
  --metrics '[{"category":"AllMetrics","enabled":true}]'
```

> **TIP** Alert on trends, not just thresholds: rising replication lag, climbing dead tuples, and storage nearing capacity give you lead time before an incident.

18. Maintenance and Upgrades

- The service auto-patches hardware, OS, and engine (minor versions).
- Choose system-managed or a custom maintenance window — set a custom window on a low-traffic day so restarts are predictable.
- Major version upgrades are in-place with pre-upgrade validation checks that flag blocking issues first.

```
# Set a custom maintenance window (e.g., Sunday 02:00)
az postgres flexible-server update \
  --resource-group rg-pgsql-prod --name pg-prod-eastus2 \
  --maintenance-window Sun:02:00

# In-place major version upgrade
az postgres flexible-server upgrade \
  --resource-group rg-pgsql-prod --name pg-prod-eastus2 \
  --version 17
```

WARNING Always run the pre-upgrade validation and take an on-demand backup before a major version upgrade. Validate in a restored copy first.

Part V — Lifecycle and Governance

19. Migration

Method	Best for
Azure Database Migration Service	Managed online/offline migrations with minimal downtime.
pg_dump / pg_restore	Small-to-medium databases; simple offline moves.
Logical replication	Near-zero-downtime cutover for larger databases.

```
# Logical dump and restore
pg_dump -Fc -h source.host -U user -d salesdb -f salesdb.dump
pg_restore -h pg-prod-eastus2.postgres.database.azure.com \
  -U pgadmin -d salesdb --no-owner salesdb.dump
```

- Assess first (versions, extensions, size), then choose method by downtime tolerance.
- Validate row counts, sequences, and application behaviour before cutover.

20. Cost Management and Optimization

- Billing = compute (per vCore-hour) + storage (per GiB-month) + backup storage (per GiB-month).
- Stop/Start: compute billing stops immediately when stopped (storage still billed). Auto-restarts after 7 days.

- Reserved capacity (1- or 3-year) cuts compute rate substantially for steady workloads.
- Burstable tier for non-production; right-size vCores against actual utilisation.

> **KEY POINT** Biggest non-prod saver is Stop/Start. Billing is per full hour a server exists — a server created and deleted within five minutes still bills a full hour.

21. Automation and Infrastructure as Code

- Azure CLI and PowerShell (Az.PostgreSql module) for scripted administration.
- ARM/Bicep and Terraform for declarative, repeatable provisioning.
- REST API and SDKs (.NET, Python, Java, Go, JavaScript) plus Ansible collections for pipelines.
- pg_cron for scheduled in-database jobs (partition maintenance, cleanup, refreshes).

```
# Terraform sketch
resource "azurerm_postgresql_flexible_server" "pg" {
  name                = "pg-prod-eastus2"
  resource_group_name = azurerm_resource_group.rg.name
  location            = "eastus2"
  version             = "16"
  sku_name            = "GP_Standard_D4ds_v5"
  storage_mb          = 131072
  administrator_login = "pgadmin"
  high_availability { mode = "ZoneRedundant" }
}
```

22. Compliance and Governance

- Audit logging via pgAudit with logs shipped to Log Analytics for retention and evidence.
- Azure Policy enforces configuration guardrails (e.g., require private access, CMK, TLS version).
- Separate Azure RBAC (control plane — who can manage the server) from database GRANT/REVOKE (data plane — who can touch data).
- Alignment aids: data residency by region, FIPS 140-2 encryption, TLS 1.2+, CMK for key control.

> **NOTE** Control-plane access (Azure RBAC) and data-plane access (database roles) are independent. A user can own the Azure resource yet have no database privileges, and vice versa.

23. Advanced and Emerging Capabilities

- Elastic clusters — horizontal scale-out / sharding for very large, high-throughput workloads.
- Azure HorizonDB — newer high-scale Postgres offering (public preview).
- Fabric mirroring — replicate Postgres data into Microsoft Fabric for analytics.
- pgvector — vector search for AI / embedding workloads.
- pg_duckdb — hybrid OLTP/OLAP, running analytical queries directly in Postgres (preview).

> **TIP** Preview features change fast. Confirm GA status and regional availability in the Azure release notes before committing production workloads.

End of document